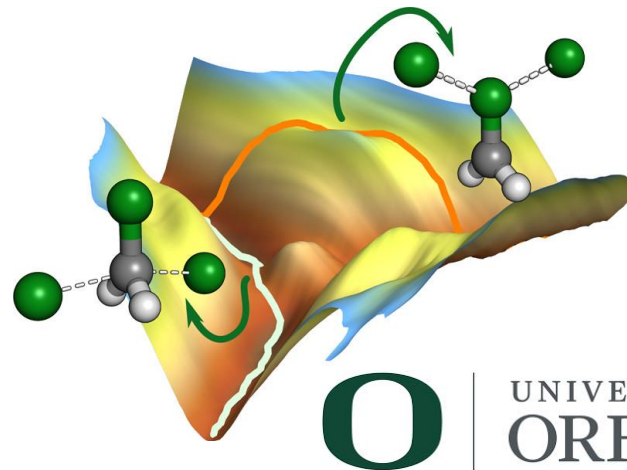
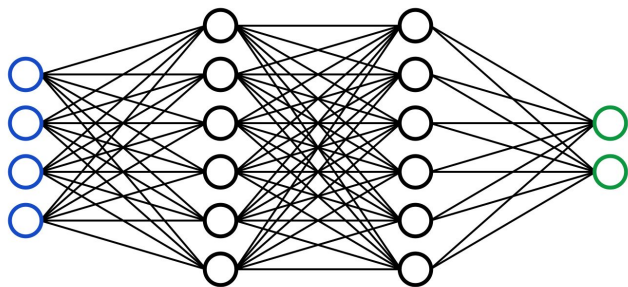


Machine Learning for Chemistry

Winter 2025

Advanced Topic 2: Graph Neural Network



Instructor: Dr. Dhiman Ray

Assistant Professor, Department of Chemistry and Biochemistry

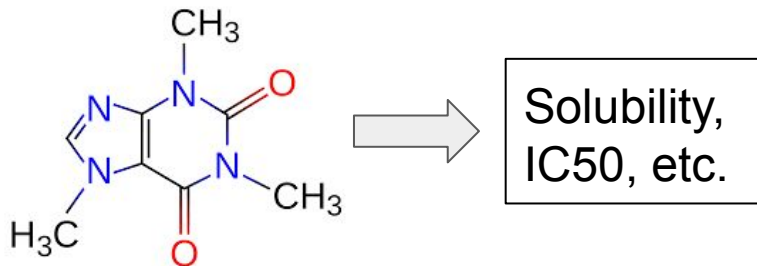
Overview of Topics

- Graph Convolution
- Graph Neural Network Architecture
- Applications in Chemistry

References:

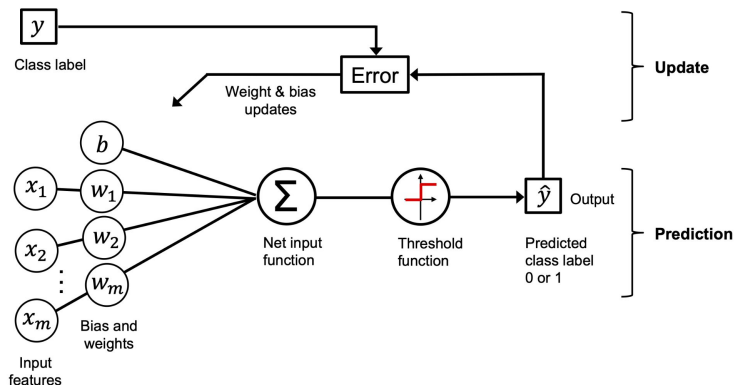
1. **Machine Learning with Pytorch and Scikit-Learn** by Raschka, Liu, and Mirjalili (Chapter 18)
2. Interactive tutorial: <https://distill.pub/2021/gnn-intro/>
3. Understanding Graph Neural Networks. Part 1, 2, and 3. DeepFindr YouTube channel: https://youtu.be/fOctJB4kVIM?si=6gb3JcOh9Nxzra_a
4. <https://colab.research.google.com/drive/16GBgwYR2ECiXVxA1BoLxYshKczNMeEAQ?usp=sharing#scrollTo=cSTDAc0jZTON>

Challenges in using Molecular Structure as input



We want to predict properties from molecular structure. But it is difficult to supply the structure as input.

- A cartesian coordinate based representation is not translationally or rotationally invariant.
- The feature space will change if you change the number (and type) of atoms, so we cannot apply the model trained on one molecule to another molecule.

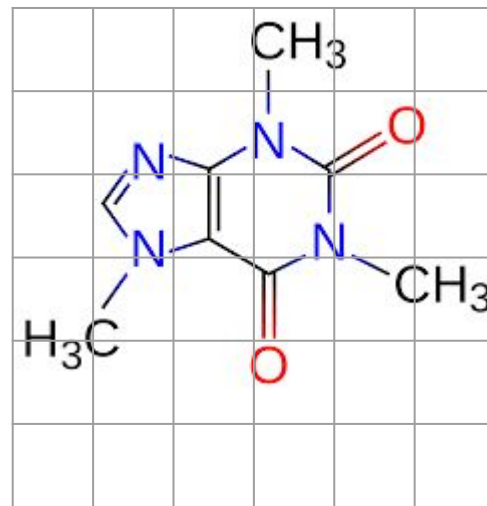
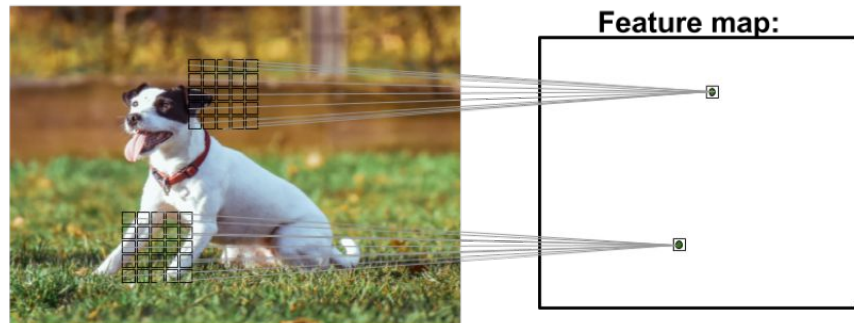


Lessons from Convolution

We learnt that for image classification, **we should not only look at the individual pixels but also its neighbors.**

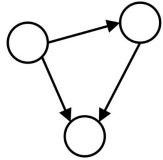
The same is true for molecules. **The property of a molecule not only depends on the individual atoms but also their neighboring atoms**, with which they are bonded.

But the structure of a molecule is not an uniform array of pixels or atoms and the connectivity is relatively complex.

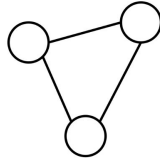


Graph Representation of Molecule

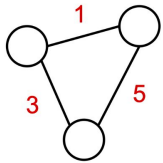
Directed graph



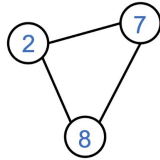
Undirected graph



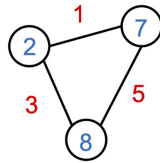
Edge-labeled
undirected graph



Node-labeled
undirected graph

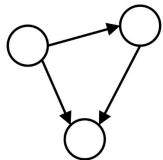


Node- and edge-labeled
undirected graph

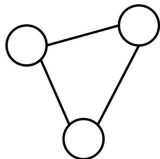


Graph Representation of Molecule

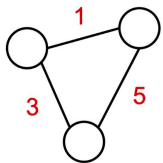
Directed graph



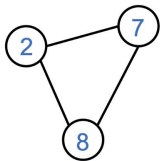
Undirected graph



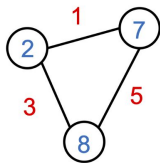
Edge-labeled
undirected graph



Node-labeled
undirected graph



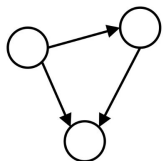
Node- and edge-labeled
undirected graph



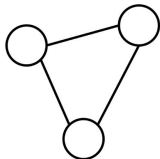
Blue: atom types. Red: bond types

Graph Representation of Molecule

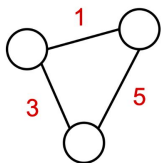
Directed graph



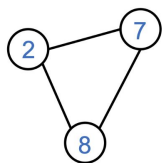
Undirected graph



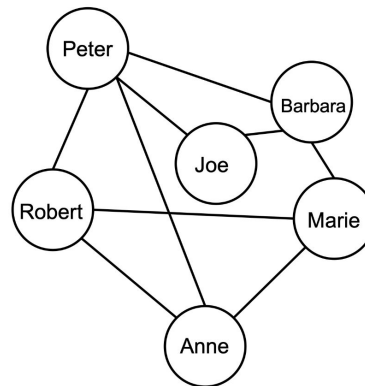
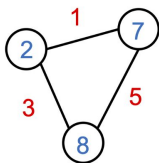
Edge-labeled
undirected graph



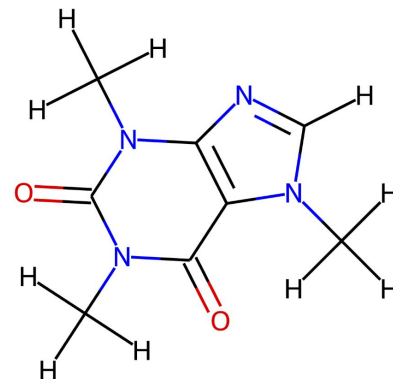
Node-labeled
undirected graph



Node- and edge-labeled
undirected graph



Friend graph

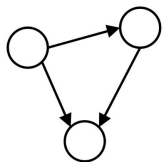


Molecular graph
of caffeine

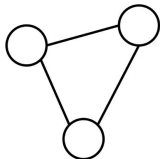
Blue: atom types. Red: bond types

Graph Representation of Molecule

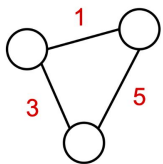
Directed graph



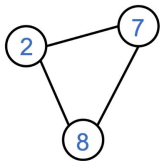
Undirected graph



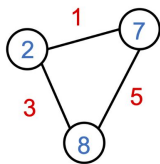
Edge-labeled
undirected graph



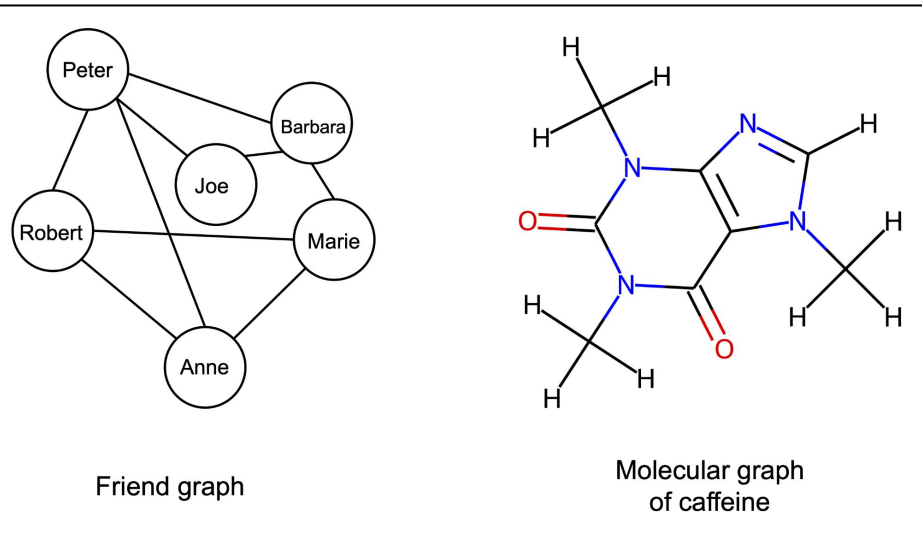
Node-labeled
undirected graph



Node- and edge-labeled
undirected graph



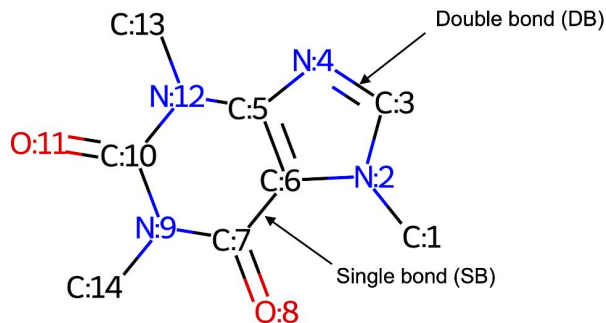
Blue: atom types. Red: bond types



How to feed this geometric structure into machine learning models?

Graph Representation of Molecule

Caffeine molecule



One-hot encoding
of the atom type

$$X = \begin{matrix} & \text{C} & \text{N} & \text{O} \\ \begin{matrix} \text{C:1} \\ \text{C:2} \\ \text{C:3} \\ \text{C:4} \\ \text{C:5} \\ \text{C:6} \\ \text{C:7} \\ \text{C:8} \\ \text{C:9} \\ \text{C:10} \\ \text{C:11} \\ \text{C:12} \\ \text{C:13} \\ \text{C:14} \end{matrix} & \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \end{matrix}$$

$X =$
Node label
matrix

One-hot encoding
of the bond type

$$\begin{matrix} & \text{SB} & \text{DB} \\ \begin{matrix} (1,2) \\ (2,3) \\ (2,6) \\ (3,4) \\ (4,5) \\ (5,6) \\ (5,12) \\ (6,7) \\ (7,8) \\ (7,9) \\ (9,10) \\ (9,14) \\ (10,11) \\ (10,12) \\ (12,13) \end{matrix} & \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 0 \end{bmatrix} \end{matrix}$$

$X_E =$
Edge label
matrix

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$A =$
Adjacency
matrix

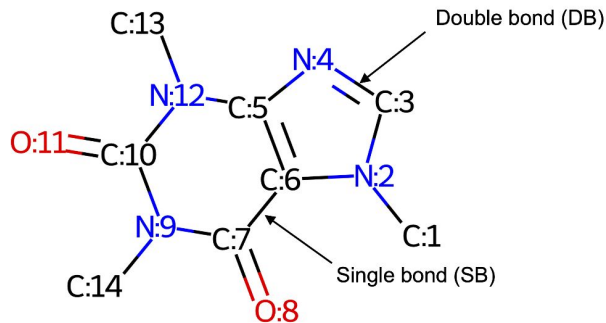
Adjacency matrix:
connectivity between
atoms

Edge label matrix:
single vs double bond

Node label matrix:
atom types.

Graph Representation of Molecule

Caffeine molecule



One-hot encoding
of the atom type

C	N	O
1	0	0
0	1	0
1	0	0
0	1	0
1	0	0
1	0	0
1	0	0
0	0	1
0	1	0
1	0	0
0	0	1
0	1	0
1	0	0

$X =$
Node label
matrix

$X_E =$
Edge label
matrix

One-hot encoding
of the bond type

	SB	DB
(1,2)	1	0
(2,3)	1	0
(2,6)	1	0
(3,4)	0	1
(4,5)	1	0
(5,6)	0	1
(5,12)	1	0
(6,7)	1	0
(7,8)	0	1
(7,9)	1	0
(9,10)	1	0
(9,14)	1	0
(10,11)	0	1
(10,12)	1	0
(12,13)	1	0

$A =$
Adjacency
matrix

0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0	0	0	0	0
0	1	0	1	0	0	0	0	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	1	0	0	0	0	0	0	1	0	0
0	1	0	0	1	0	1	0	0	0	0	0	0	0	0
0	0	0	0	0	1	0	1	1	0	0	0	0	0	0
0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0	1	0	1	1	0	0	0
0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	0	0	1	0	0
0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
0	0	0	0	0	0	0	0	1	0	0	0	0	0	0

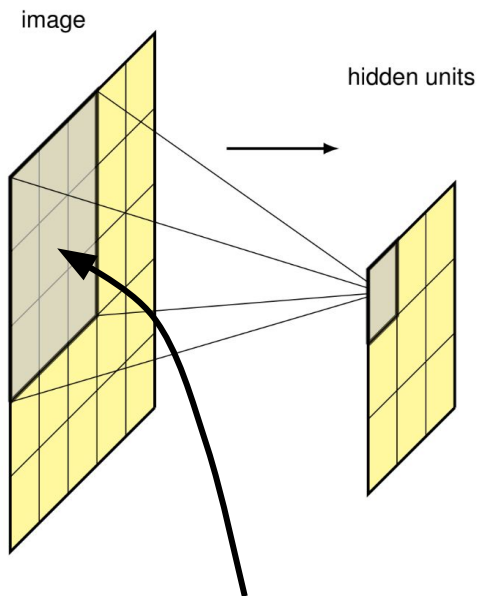
Advantages

Efficiently use the atom connectivity information.

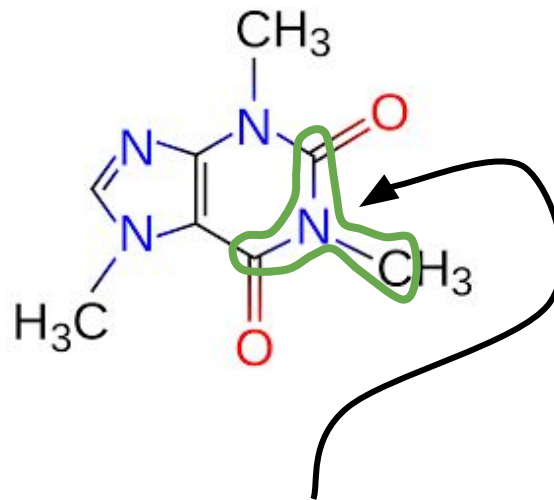
Rotational, translational, and permutation invariance.

Convolution for graphs can function with a fixed parameter set for graphs of different sizes. Most graphs including molecules contain graphs of varying sizes.

Philosophy of Convolution

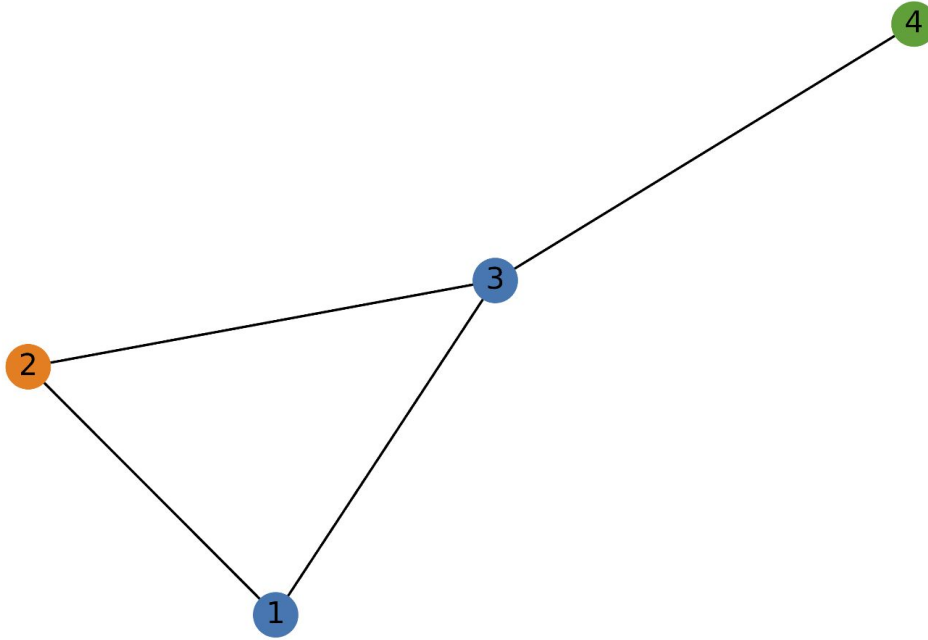


A pixel in an image makes sense in the context of its neighboring pixels.



An atom in a molecule makes sense only in the context of its neighboring connected atoms.

Graph Convolution

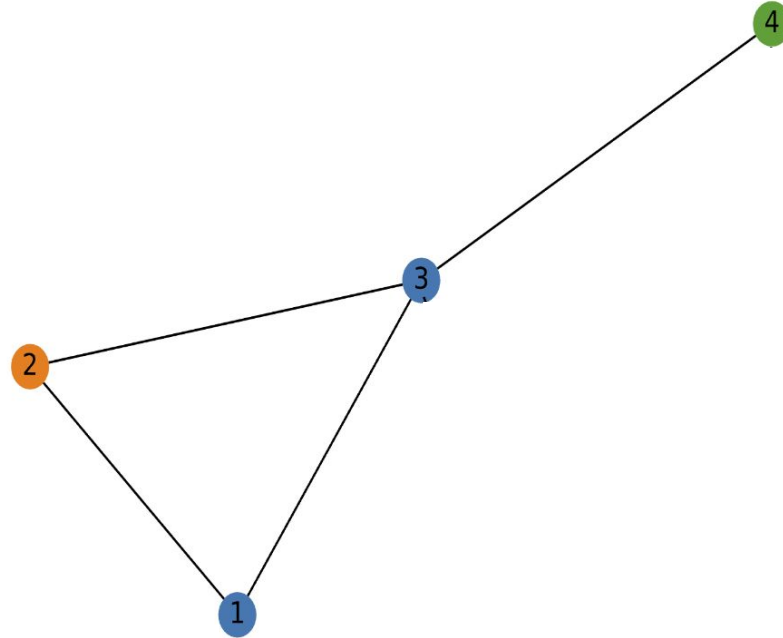


$$X = \begin{matrix} \text{G} & \text{B} & \text{O} \\ \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \end{matrix}$$

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

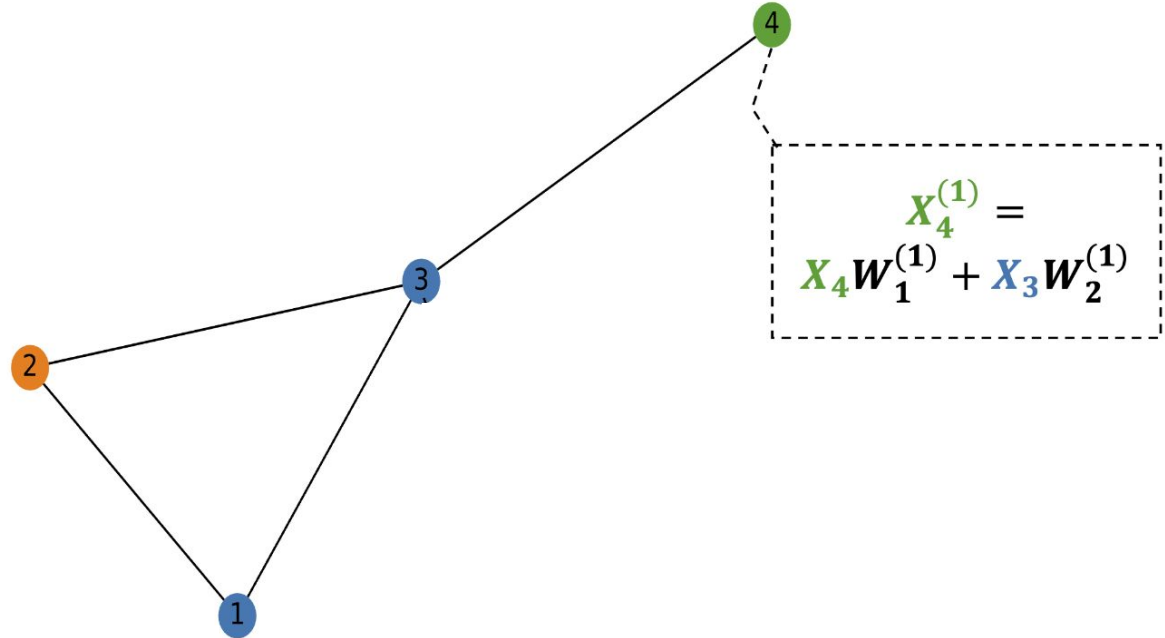
Graph Convolution

Node embeddings after 1st graph convolution



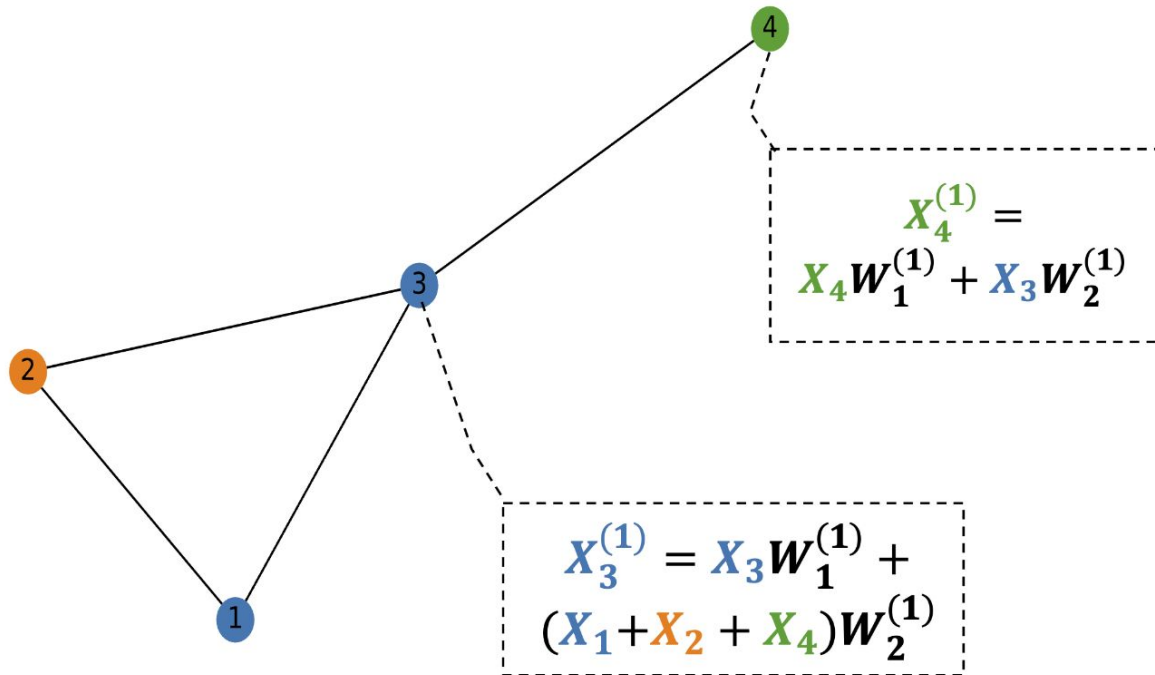
Graph Convolution

Node embeddings after 1st graph convolution



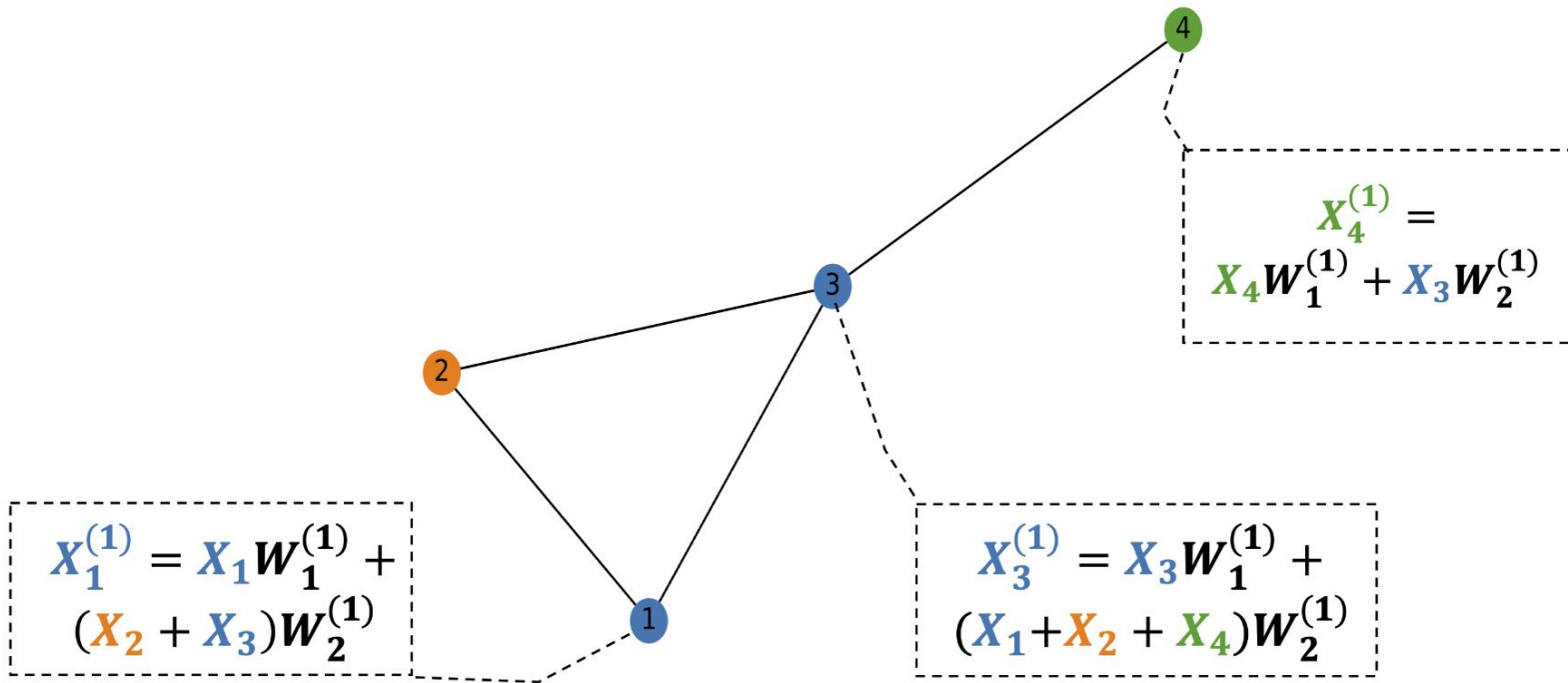
Graph Convolution

Node embeddings after 1st graph convolution



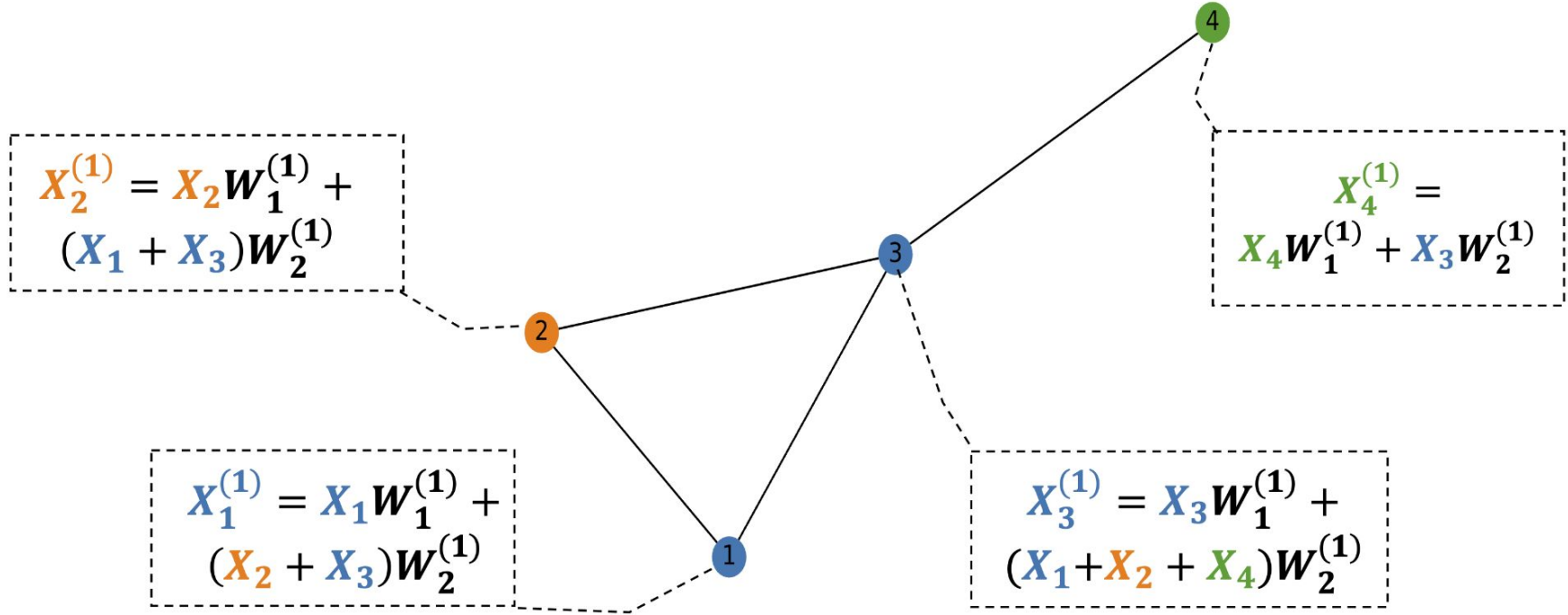
Graph Convolution

Node embeddings after 1st graph convolution

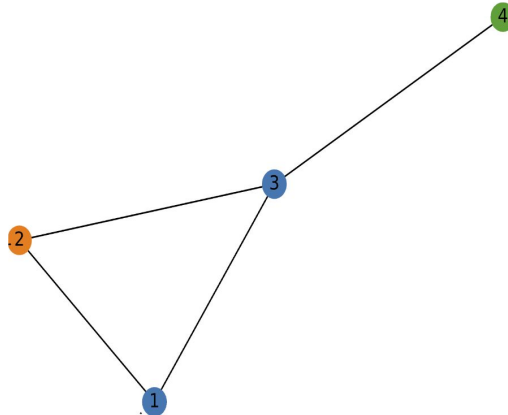


Graph Convolution

Node embeddings after 1st graph convolution



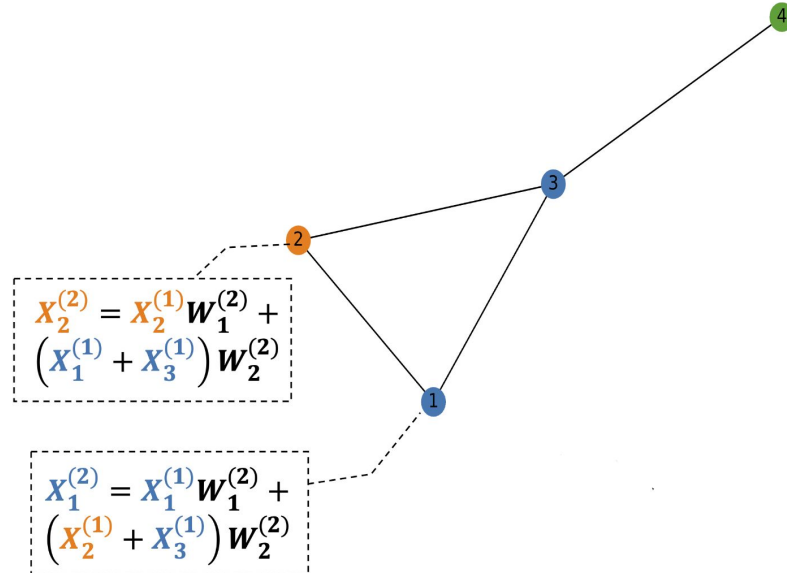
Graph Convolution



Node embeddings after
2nd graph convolution

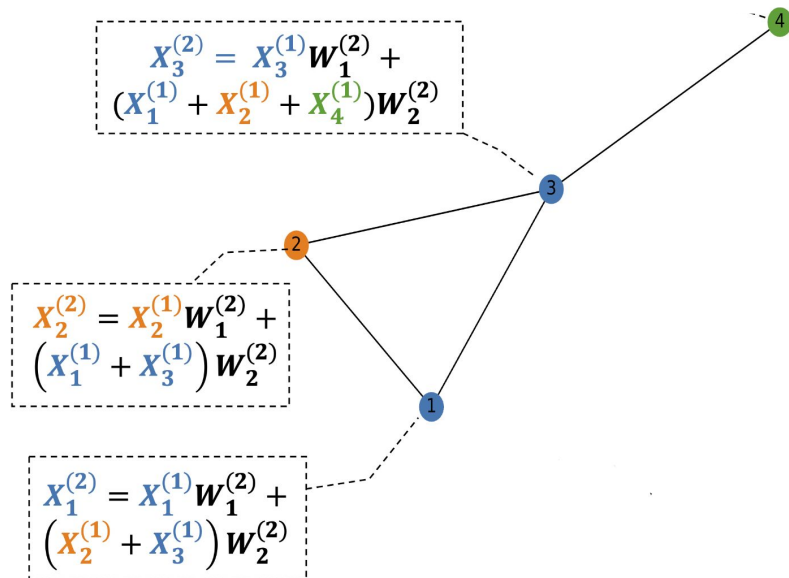
$$X_1^{(2)} = X_1^{(1)} W_1^{(2)} + (X_2^{(1)} + X_3^{(1)}) W_2^{(2)}$$

Graph Convolution



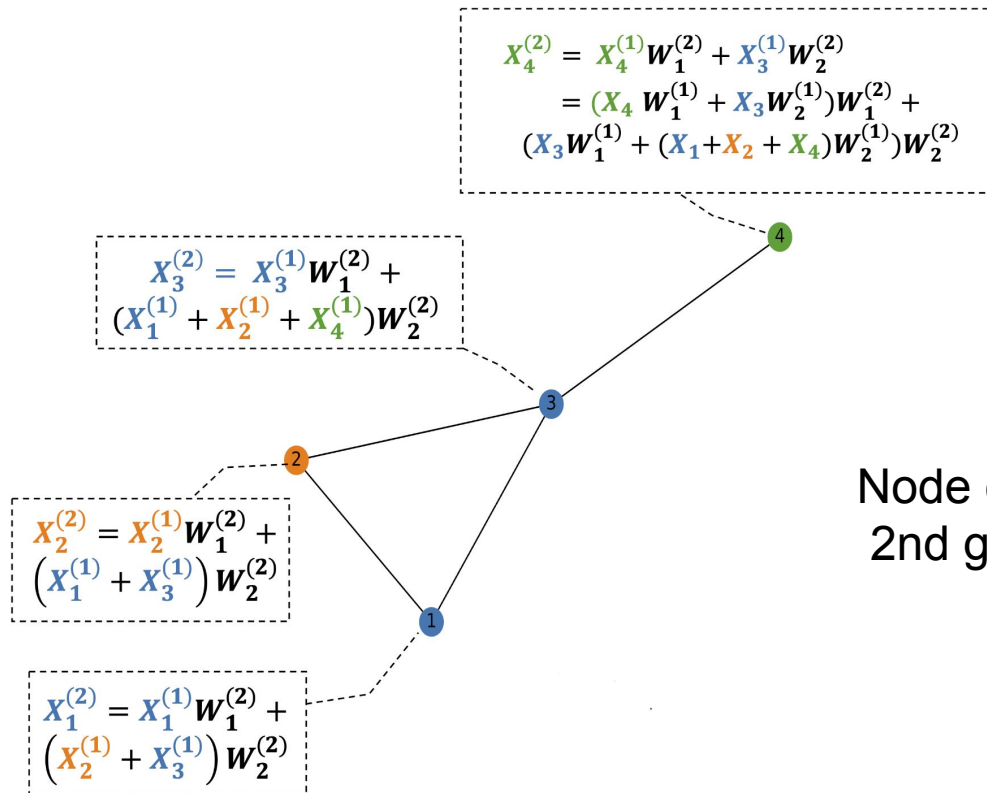
Node embeddings after
2nd graph convolution

Graph Convolution



Node embeddings after
2nd graph convolution

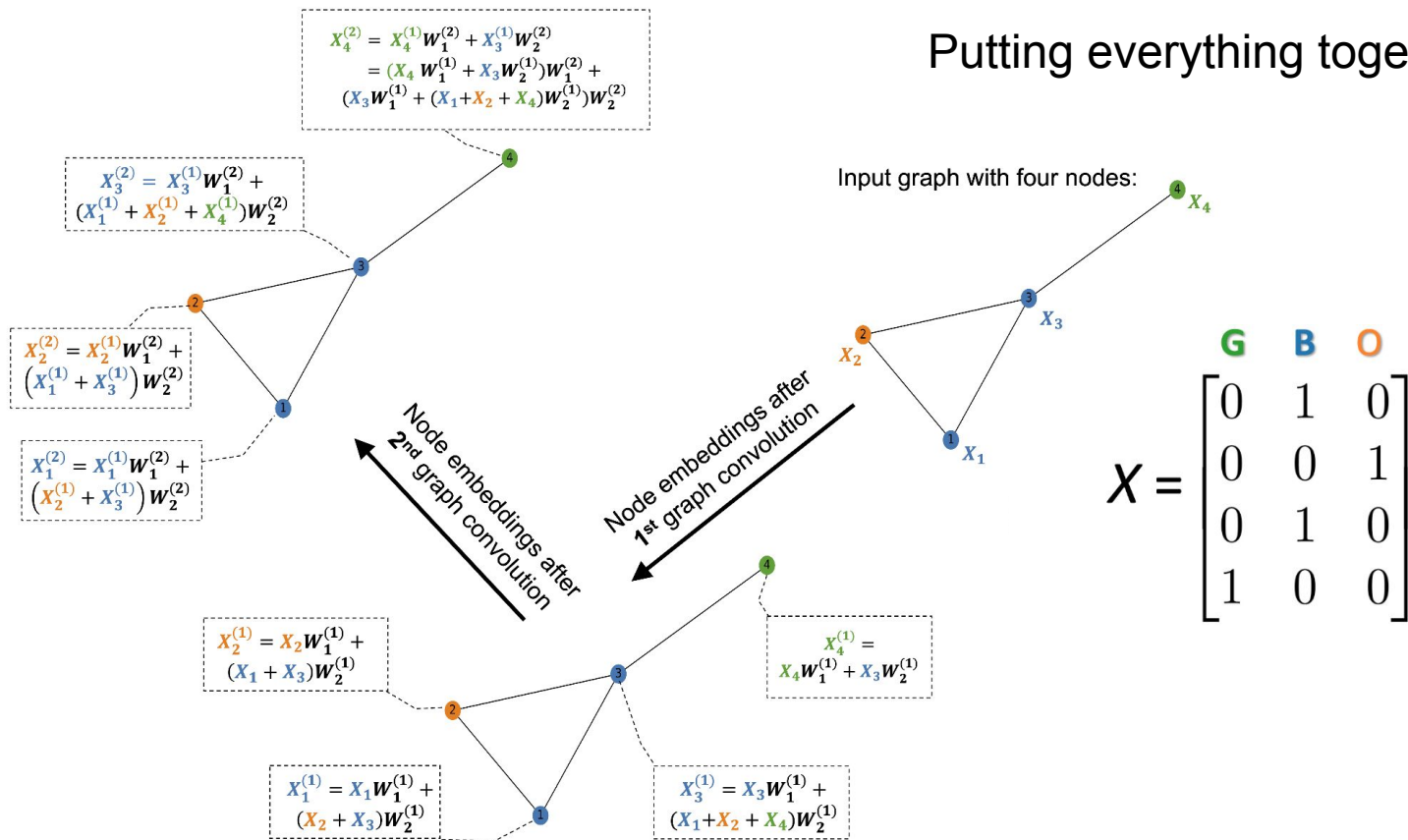
Graph Convolution



Node embeddings after
2nd graph convolution

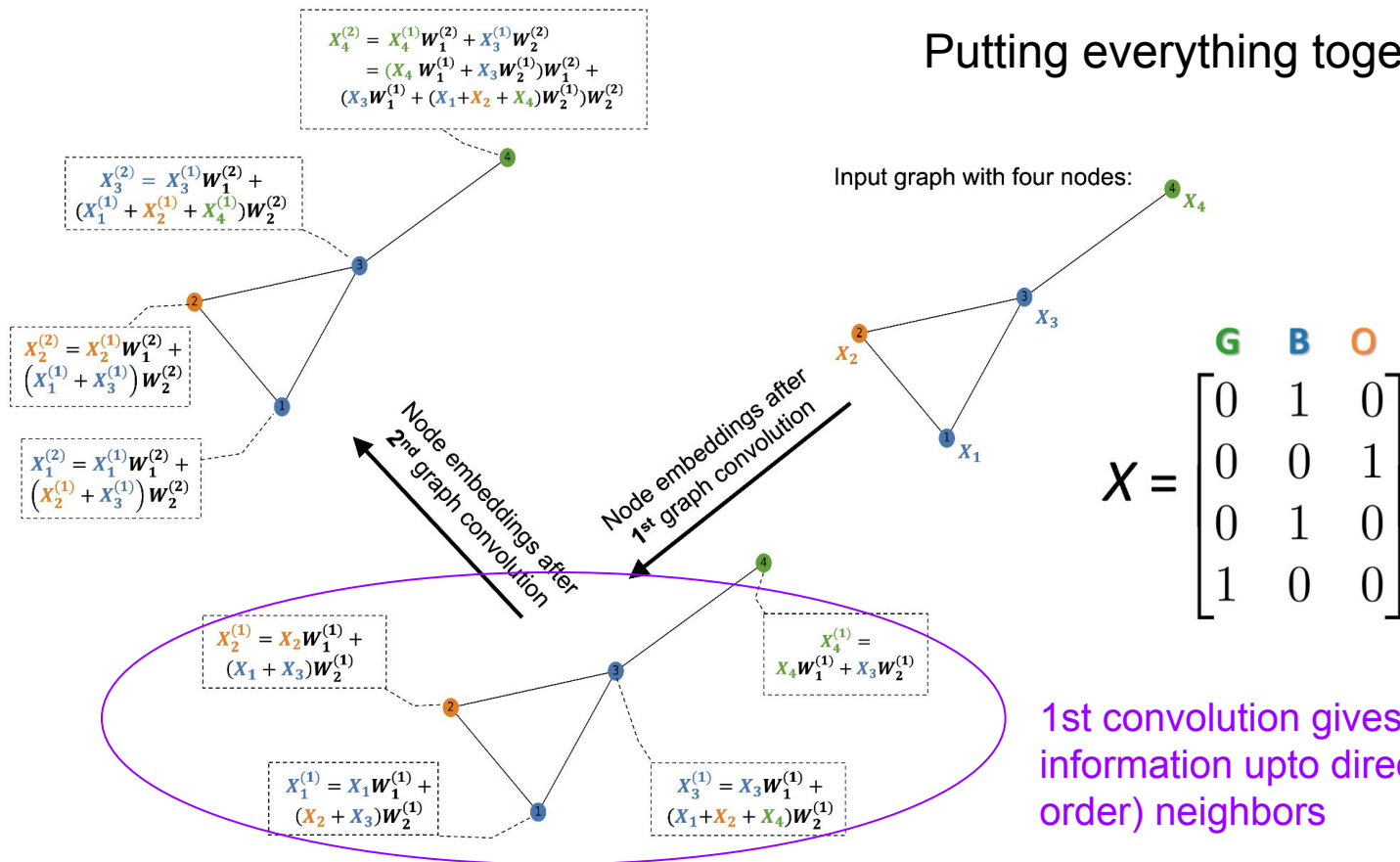
Graph Convolution

Putting everything together



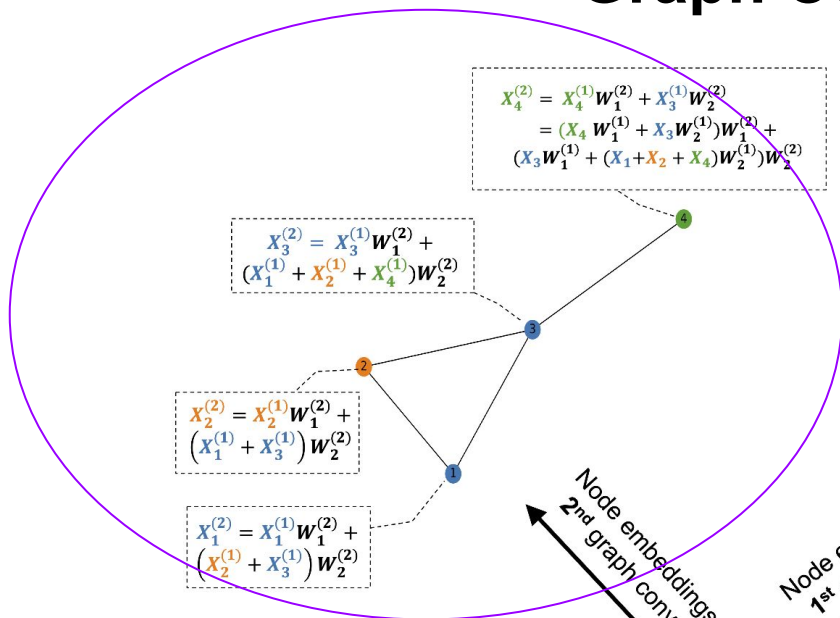
Graph Convolution

Putting everything together



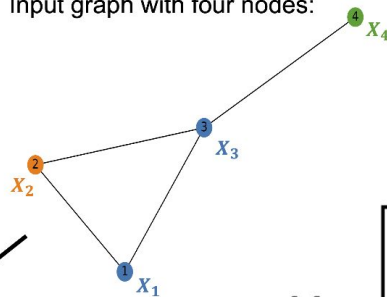
Graph Convolution

Putting everything together

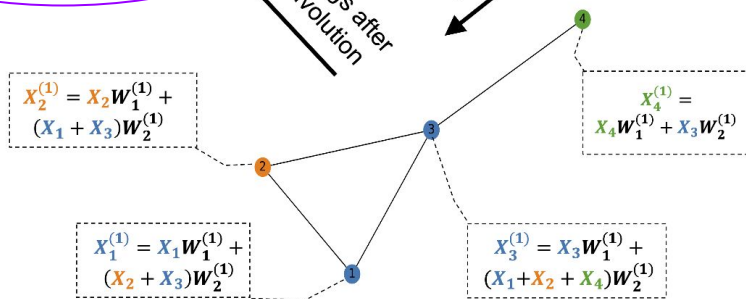


2nd convolution gives information upto secondary neighbors

Input graph with four nodes:

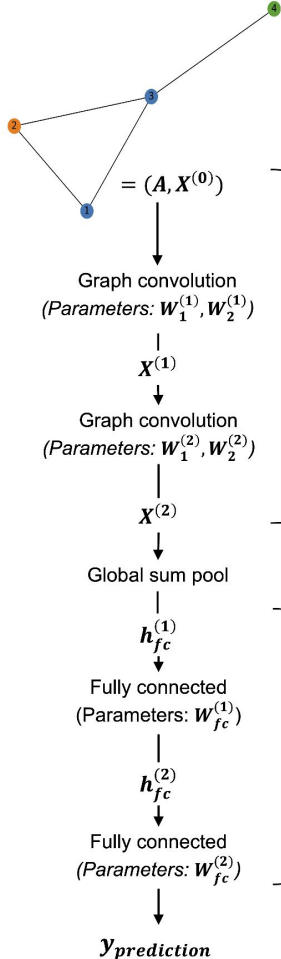


Node embeddings after 1st graph convolution

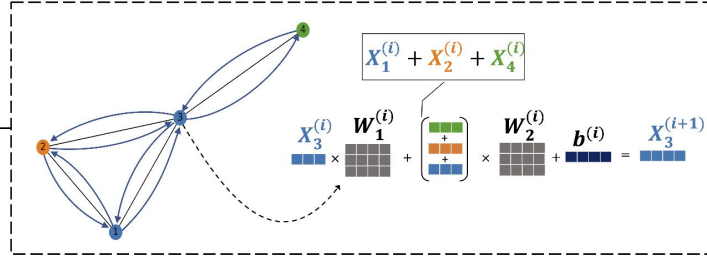


$$X = \begin{bmatrix} \text{G} & \text{B} & \text{O} \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

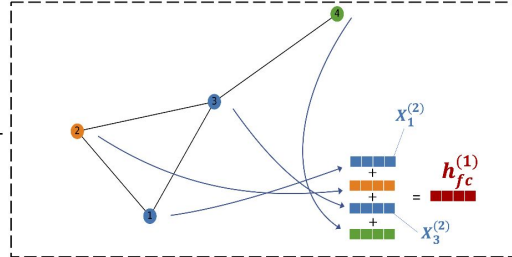
Graph Neural Network Architecture



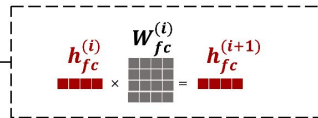
Graph convolution layer:



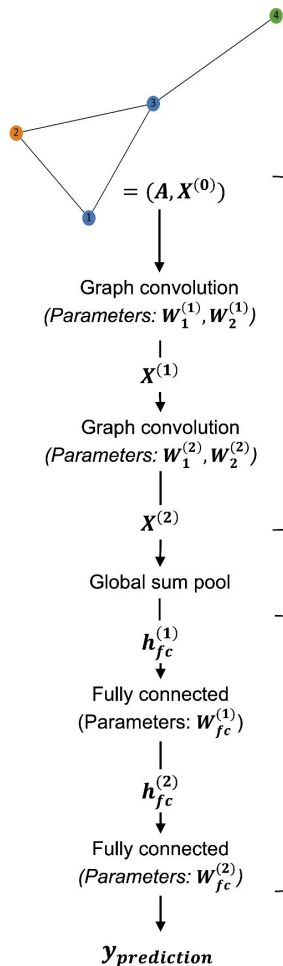
Global sum pooling layer:



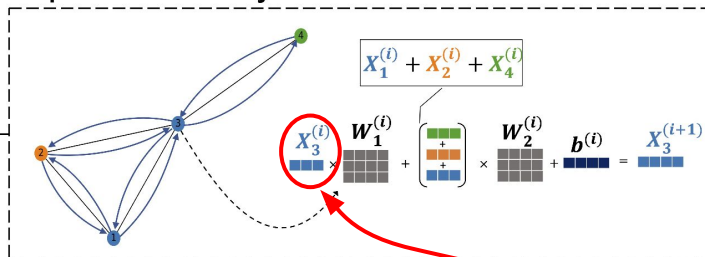
Fully connected layer:



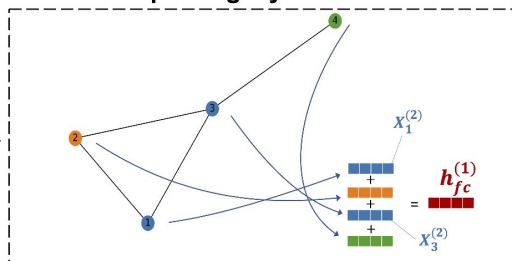
Graph Neural Network Architecture



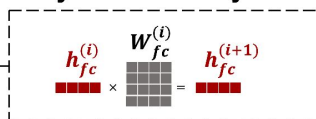
Graph convolution layer:



Global sum pooling layer:



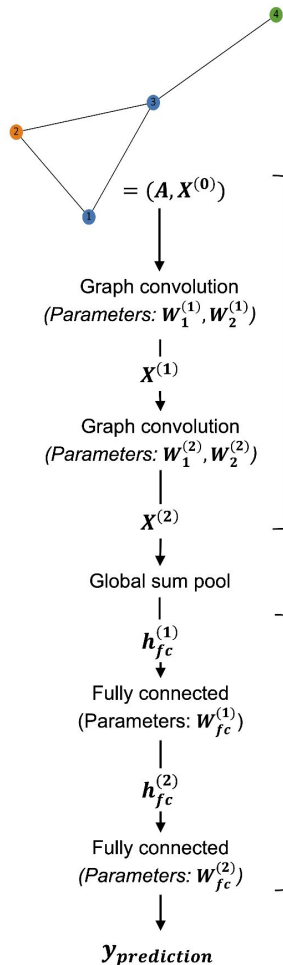
Fully connected layer:



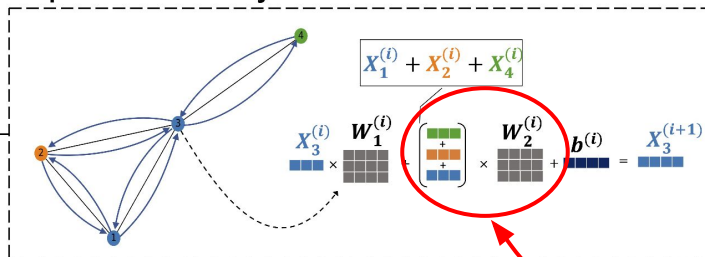
Information about the 3rd atom. The size of the input vector (3) is system dependent. There are 3 kinds of atoms: blue, orange, and green

$$X = \begin{matrix} & \text{G} & \text{B} & \text{O} \\ \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \end{matrix}$$

Graph Neural Network Architecture

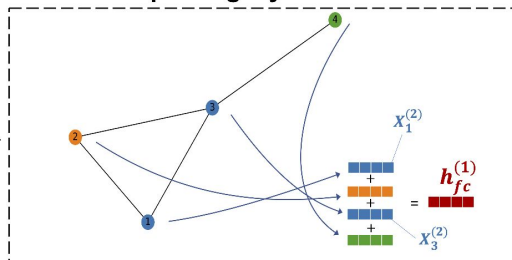


Graph convolution layer:

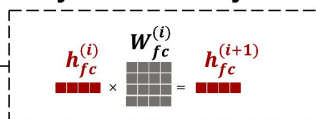


Combining information about neighboring atoms

Global sum pooling layer:

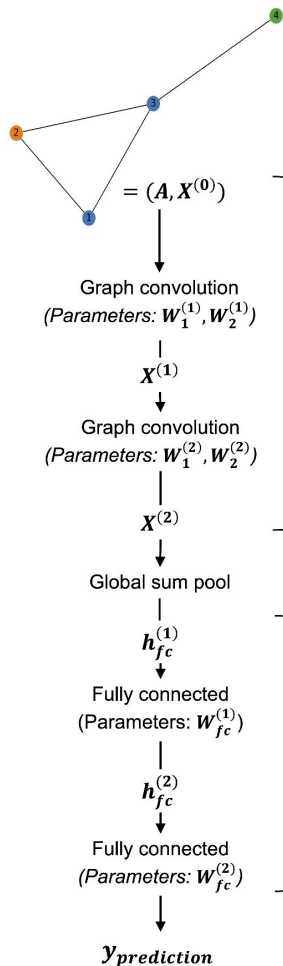


Fully connected layer:

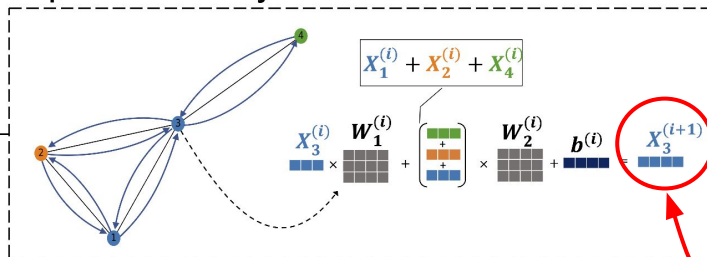


$$X_3^{(1)} = X_3 W_1^{(1)} + (X_1 + X_2 + X_4) W_2^{(1)}$$

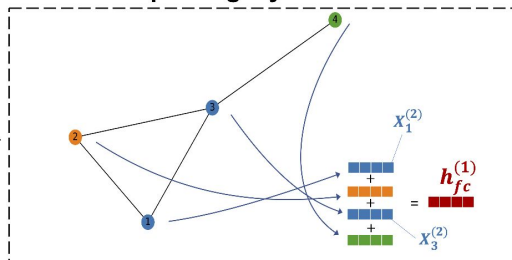
Graph Neural Network Architecture



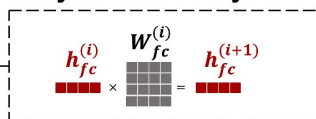
Graph convolution layer:



Global sum pooling layer:



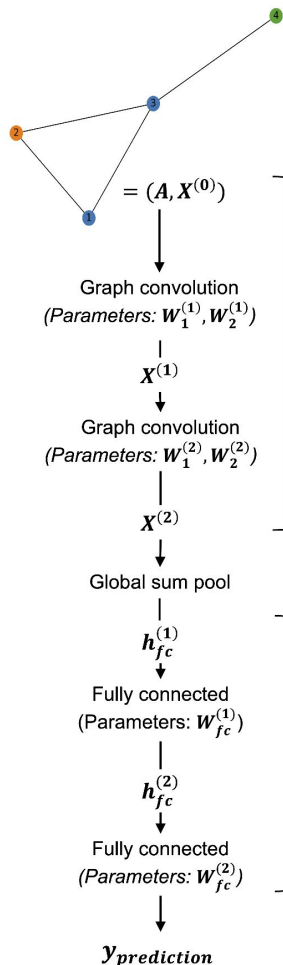
Fully connected layer:



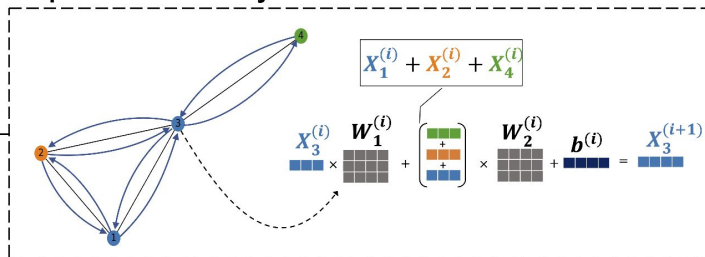
Convolved (embedded) input for the atom. Note the dimensionality is no longer 3. The current dimensionality (4) is based on our choice

$$X_3^{(1)} = X_3 W_1^{(1)} + (X_1 + X_2 + X_4) W_2^{(1)}$$

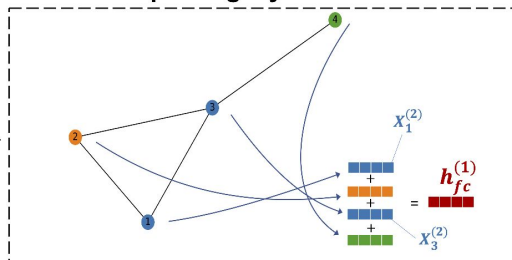
Graph Neural Network Architecture



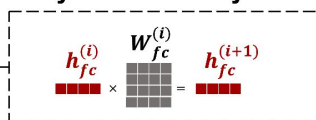
Graph convolution layer:



Global sum pooling layer:



Fully connected layer:

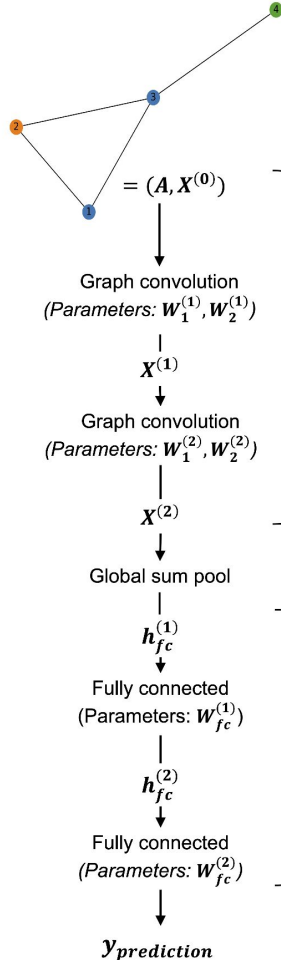


We need a feature space that does not depend on number of atoms (so that our network can be applied to different molecules).

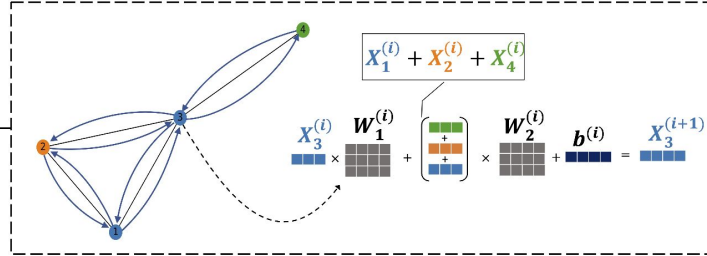
Global pooling combines the embedded descriptors for all the atoms.

The dimensionality after pooling is 4 which, as mentioned, is our choice and not system dependent.

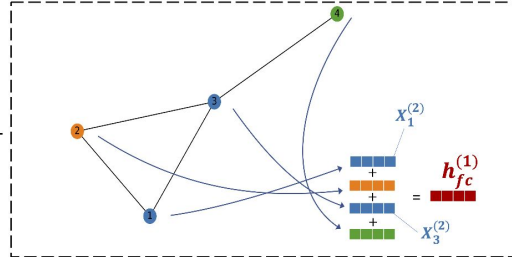
Graph Neural Network Architecture



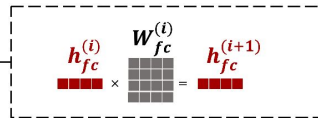
Graph convolution layer:



Global sum pooling layer:



Fully connected layer:



Train a feedforward neural network on the embedded and pooled feature space.

Implementation in PyTorch with torch_geometric

```
import torch
from torch.nn import Linear
import torch.nn.functional as F
from torch_geometric.nn import GCNConv, TopKPooling, global_mean_pool
from torch_geometric.nn import global_mean_pool as gap, global_max_pool as gmp
embedding_size = 64

class GCN(torch.nn.Module):
    def __init__(self):
        # Init parent
        super(GCN, self).__init__()
        torch.manual_seed(42)

        # GCN layers
        self.initial_conv = GCNConv(data.num_features, embedding_size)
        self.conv1 = GCNConv(embedding_size, embedding_size)
        self.conv2 = GCNConv(embedding_size, embedding_size)
        self.conv3 = GCNConv(embedding_size, embedding_size)

        # Output layer
        self.out = Linear(embedding_size*2, 1)

    def forward(self, x, edge_index, batch_index):
        # First Conv layer
        hidden = self.initial_conv(x, edge_index)
        hidden = F.tanh(hidden)

        # Other Conv layers
        hidden = self.conv1(hidden, edge_index)
        hidden = F.tanh(hidden)
        hidden = self.conv2(hidden, edge_index)
        hidden = F.tanh(hidden)
        hidden = self.conv3(hidden, edge_index)
        hidden = F.tanh(hidden)

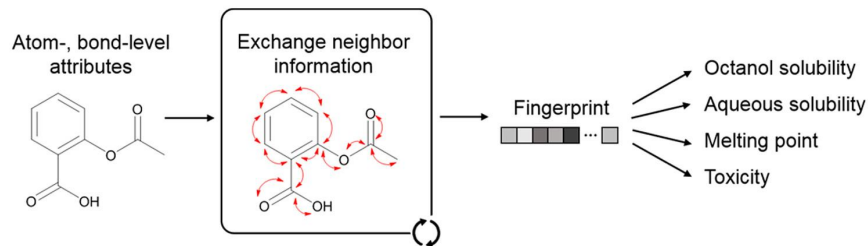
        # Global Pooling (stack different aggregations)
        hidden = torch.cat([gmp(hidden, batch_index),
                           gap(hidden, batch_index)], dim=1)

        # Apply a final (linear) classifier.
        out = self.out(hidden)

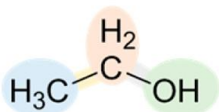
        return out, hidden
```

1. Perform four graph convolutions (`self.initial_conv`, `self.conv1`, `self.conv2`, and `self.conv3`)
2. Pool all the node embeddings via `global_mean_pool`, and `global_max_pool`, concatenated.
3. Run the pooled embeddings through a linear layer (`self.out`). This can be replaced by a fully connected neural network if necessary
4. Return the output

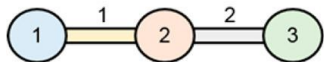
Applications in Chemistry



Structure



Attributed Graph



	Z	# neighbors	# hydrogens	Formal charge
Node 1	6	1	3	0
Node 2	6	2	2	0
Node 3	8	1	1	0

	Order	Aromatic	Conjugated	In ring	Connects
Edge 1	Single	No	No	No	(1, 2)
Edge 2	Single	No	No	No	(2, 3)

$$M_{\text{ethanol}} = \begin{bmatrix} 6, 1, 3, 0, 0, 0, 0, 0, 0 & 6, 2, 2, 0, 1, 0, 0, 0, 1 & 0, 0, 0, 0, 0, 0, 0, 0, 0 \\ 6, 1, 3, 0, 1, 0, 0, 0, 1 & 6, 2, 2, 0, 0, 0, 0, 0, 0 & 8, 1, 1, 0, 1, 0, 0, 0, 1 \\ 0, 0, 0, 0, 0, 0, 0, 0, 0 & 6, 2, 2, 0, 1, 0, 0, 0, 1 & 8, 1, 1, 0, 0, 0, 0, 0, 0 \end{bmatrix}$$

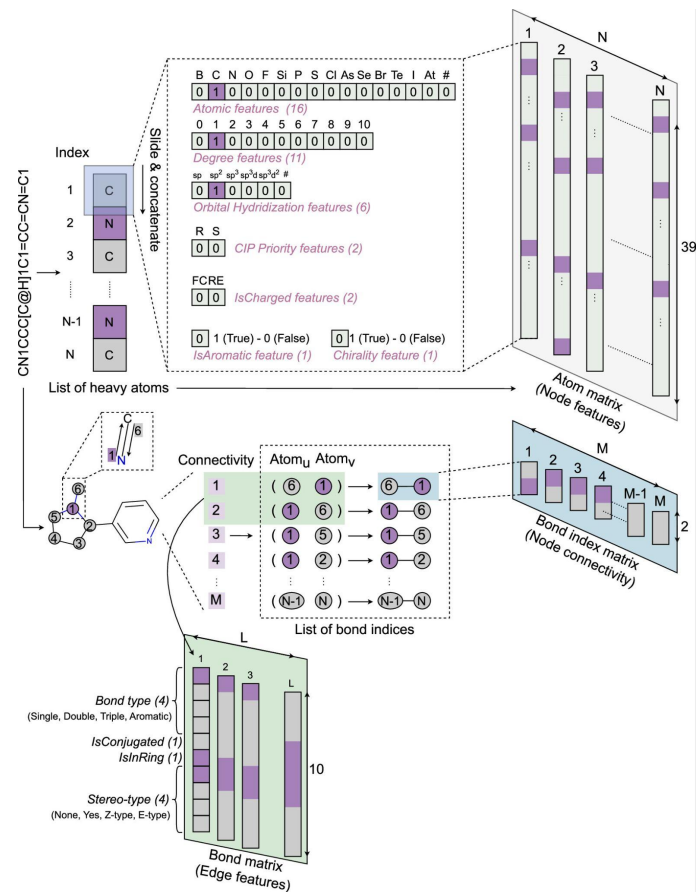
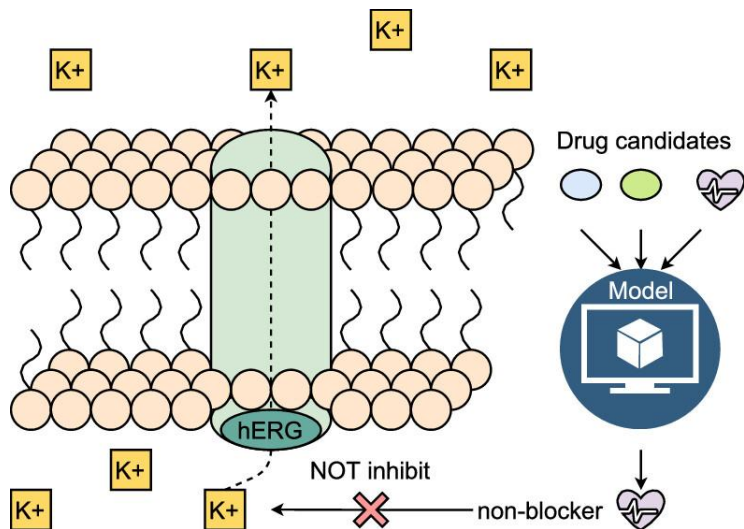
Convolutional Embedding of Attributed Molecular Graphs for Physical Property Prediction

Connor W. Coley, Regina Barzilay, William H. Green, Tommi S. Jaakkola, and Klavs F. Jensen

Journal of Chemical Information and Modeling 2017 57 (8), 1757-1772

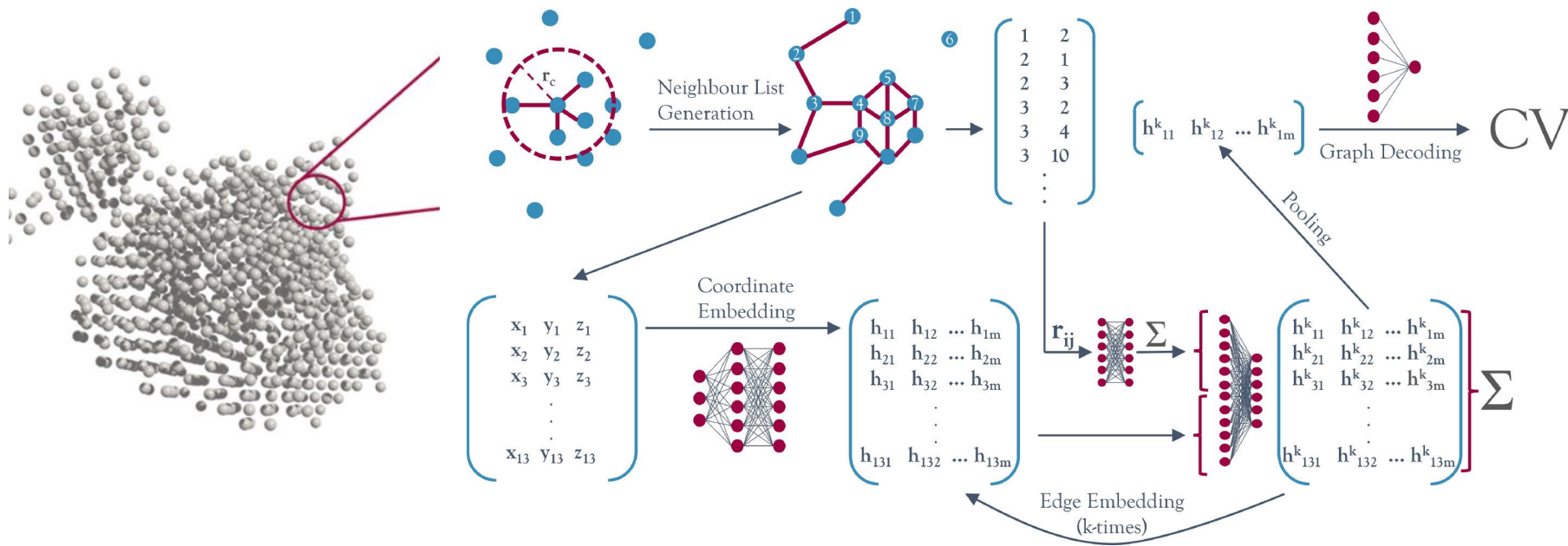
DOI: 10.1021/acs.jcim.6b00601

Applications in Chemistry



Predicting Cardiotoxicity of Molecules Using Attention-Based Graph Neural Networks Tuan Vinh, Loc Nguyen, Quang H. Trinh, Thanh-Hoang Nguyen-Vo, and Binh P. Nguyen *Journal of Chemical Information and Modeling* 2024 64 (6), 1816-1827
DOI: 10.1021/acs.jcim.3c01286

Applications in Chemistry



Machine Learning Nucleation Collective Variables with Graph Neural Networks

Florian M. Dietrich, Xavier R. Advincula, Gianpaolo Gobbo, Michael A. Bellucci, and Matteo Salvalaglio *Journal of Chemical Theory and Computation* 2024 20 (4), 1600-1611 DOI: 10.1021/acs.jctc.3c00722